

CS293 Project

Dhanush Sanjay Nidamanuri-24B0968

Vivekanand Siddharthi-24B1056

Tagore Ravindra Sree-24B1041

November 2025

Chapter 1

Directory Structure

1.1 Main directory

```
Lazarus\_CS293/
├── Phase-1/                (Contains all contents related to phase 1)
│   ├── testcases/
│   └── ...
├── Phase-2/                (Contains all contents related to phase 2)
│   ├── testcases/
│   └── ...
├── Phase-3/                (Contains all contents related to phase 3)
│   ├── testcases/
│   └── ...
├── makefile                (Creates the executables and the testcases for each phase)
├── approx_path_verify.py  ()
├── Report.pdf
├── LICENSE
├── .gitignore              (To ensure not committing temporary files)
└── README.md              (Contains summary)
```

1.2 Sub-directories

1. Phase-1:

```
Phase-1/
├── graph.hpp               (Declaration of the graph class and its methods)
├── graph.cpp               (Implementations of graph methods)
├── ShortestPath.hpp        (Declaration of solution and result class)
├── ShortestPath.cpp        (Implementations of solution class's methods)
├── KNN.hpp                 (Declaration of solution and result class)
├── KNN.cpp                 (Implementations of solution class's methods)
├── main.cpp                (Driver which integrates all the files and runs tests)
├── testcase_generator.py   (Script that parses and generates query json from API's)
├── testcases/
│   ├── queries_test1.json  (Query test file)
│   └── test1.json          (Graph testcase)
```

2. Phase-2:

Phase-2/

— Approx_Shortest.hpp	(Declaration of approx shortest path solver and result)
— Approx_Shortest.cpp	(Implementations of approximate shortest path methods)
— graph.hpp	(Declaration of graph class and methods)
— graph.cpp	(Implementation of graph methods)
— K_Shortest.hpp	(Declaration of k shortest paths solver & result classes)
— K_Shortest.cpp	(Implementation of k shortest paths solver methods)
— main.cpp	(Driver that integrates all files and runs tests)
— testcase_generator.py	(Script that parses and generates query json from API's)
— testcases/	
— queries_test1.json	(Query test file)
— test1.json	(Graph testcase)

3. Phase-3:

Phase-3/

— graph.hpp	(Declaration of the graph class and its methods)
— graph.cpp	(Implementations of graph methods)
— main.cpp	(Driver that integrates all files and runs tests)
— tsp.hpp	(Declaration of classes Order, DriverRoute, Solution etc)
— tsp.cpp	(Implementations of the above classes' methods)
— testcase_generator.py	(Script that parses and generates query json from API's)
— testcases/	
— queries_test1.json	(Query test file)
— test1.json	(Graph testcase)

Chapter 2

Makefile explanation

For each phase i , we have 4 different targets.

1. **$\$(P\{i\}_OBJ_DIR)/\%.o$** compiles all the .cpp files and links with corresponding .hpp file to create the object files in the current phase's directory. It also creates a directory to store those object files if not present already.
2. **$phase\{i\}_notest$** links all the object files to prepare the final executable for the phase.
3. **$P\{i\}_generate_testcases$** runs the testcase generator python script, creating the json test files.
4. **$phase\{i\}$** does the same as $phase\{i\}_notest$ but also has $P\{i\}_generate_testcases$ as a dependency, making sure to rebuild the testcases every time.

And finally, the target "clean" deletes the binaries and object files for all the phases. Note: Ensure that you have c++ libraries "nlohmann/json.hpp" and "cmath" installed before compiling.

Chapter 3

Assumptions taken

- Given graph is a simple graph.
- The node id's are added to the graph in order from 0 to n-1.
- Speeds given in speed_profile are in meters per second.

Chapter 4

Solutions

4.1 Shortest path

To find the shortest path between a source and target, we have implemented an **A* search that is guided by heuristic weights** between the nodes. For each pair of nodes, we converted the coordinates into radians and then their heuristic distance is calculated by using the formula for the distance between 2 points on a sphere (using radius of earth). The heuristic time is calculated from the heuristic distance assuming a constant speed. These guide the search for their respective metric while handling cases like forbidden nodes, road types and one way roads.

4.2 K nearest neighbours

While adding the nodes to the graph in the construction, we have stored all the nodes with the same poi in a map. For the euclidean metric, all the nodes with the same poi as the given poi are **pushed into a min-heap** and then k of the min-heap entries are popped and returned as the output. For the shortest path metric, the closest distance node is computed naively by iterating through the entire graph and then the graph is subjected to a **Dijkstra search**. The dijkstra search is stopped when we get either k finalised (popped from the heap) nodes with the same poi or when we finalise all the nodes with the same poi. All the popped nodes with the same poi are returned in the same order as they were popped.

4.3 K exact shortest paths

Yen's algorithm[\[1\]](#) was used as the primary algorithm. In the Yen's loop, for each shortest path query between a source and a target, A* is used. A hashmap is stored to maintain unique paths. The intermediate set of paths obtained are stored in a min-heap.

4.4 K heuristic shortest paths

We first choose $2k$ exact shortest paths by calling the above function for exact paths. Out of these paths, only the simple paths are filtered in. A selection algorithm is applied on the resultant set which selects the k paths having the minimum score. Score function: $\text{overlap penalty (based on \% edges repeated in already selected paths)} \times \text{distance penalty}$.

4.5 Approx shortest path

We first label a set of nodes as landmarks which are produced using a scoring function. Scoring function returns $(\text{minimum distance from any of the landmarks}) / (\text{the degree of the node})$. We got 2200 isolated nodes as landmarks out of before accounting for the degree of the node, so we had to use this function. In the process, all the nodes' distances from their nearest landmarks are stored. An upper bound based on the triangle inequality is established by using the distances from the closest landmark. If the time permits, the algorithm runs A* normally while checking the passed time. Let the distance from source to its nearest landmark be LS, for target LT and the distance between those landmarks be LL. If the time

limit is almost reached, it immediately returns $(LS+LT+LL)$ if $(LL/(LS+LT))>\text{tuning}$ (its a constant arrived from geometrical analysis).

Chapter 5

Complexities

5.1 Shortest path

Time Complexity: $O((V + E) \log(V))$ with improvement depending on the heuristic
Space Complexity: $O(V + E)$

5.2 K nearest neighbours

- **KNN Euclidean:**
Let the number of nodes containing the correct poi be P .
Time Complexity: $O(P + K \log(P))$
Space Complexity: $O(\min(K, P))$
- **KNN Shortest path:**
Time Complexity: $O((V + E) \log(V))$
Space Complexity: $O(V + K)$

5.3 K exact shortest paths

Time Complexity: $O(VK(V + E) \log(V))$
Space Complexity: $O(V(V + K))$

5.4 K heuristic shortest paths

Time Complexity: $O(VK(V + E) \log(V) + K^3V)$
Space Complexity: $O(VK(V + K))$

5.5 K approx shortest paths

Time Complexity: $O(Q(V + E) \log(V))$
Space Complexity: $O(V + E + Q)$

Chapter 6

Python Scripts

For each phase, there is a **testcase_generator.py** file. Each of them can be run using the following command to get the json test files.

```
python3 testcase_generator.py
```

Internal libraries json, os, math, random, pathlib, random and external library **requests** were used in the files.

To analyse the results of approximate k shortest paths in phase 2, the file **approx_path_verify.py** was used. In addition to some of the above libraries, numpy and pandas(both external) were used in this file. It can be run by

```
python3 approx_path_verify.py
```

Chapter 7

Testcase analysis

To verify the results of approximate k shortest paths in phase 2, we have written a python script `approx_path_verify.py`. It parses the outputs for exact k shortest paths and approximate k shortest paths for the same input graph and queries. It then computes the percentage difference for each query. Finally, it outputs the largest error %, least error%, mean error % and the number of threshold violations.

```
===== SUMMARY =====
total_queries           : 12054
max_error               : 116.03909496223956
min_error               : -33.148435198999984
mean_error              : 1.5897939538076329
threshold_violations    : 164
No of approx paths given : 5827
Incorretly_No_Path_given : 0

Saved analysis_approx_output.csv
dhanushsaniya@DHANUSH SANJAY: /mnt/c/Users/dhanu/OneDrive/Edu
```

Figure 7.1: Caption

Chapter 8

AI utilisation

Here are the links to all the AI chats by our group

- <https://chatgpt.com/share/692195d0-caec-8011-a756-1c81f1e44785>
- <https://chatgpt.com/share/6921eed4-43d0-8011-840e-5435439bf271>
- <https://chatgpt.com/share/6921eee5-1704-8011-908d-6ac518cbaa25>
- <https://chatgpt.com/share/6921ef2d-87b4-8011-9ca2-e71107080918>
- <https://chatgpt.com/share/6905f378-a3b0-8011-bbbb-a17c510aaecc>
- <https://chatgpt.com/share/6921ef5d-95ec-8011-b37c-df690b7029b9>
- <https://chatgpt.com/share/6921ef75-5670-8011-9623-b857f16fae6a>
- <https://chatgpt.com/share/6921eea7-19a0-8011-9a33-5ae0afb9042c>
- <https://chat.deepseek.com/share/mc6omg2a7c7h6tqbqz>

Chapter 9

Phase 3

This TSP module assigns pickup–dropoff orders to multiple drivers using a greedy, distance-based heuristic over a precomputed all-pairs shortest-path (APSP) matrix. First, all orders are sorted by how far their pickup is from the depot, ensuring that nearby tasks are allocated earlier. Each order is assigned to the driver whose current route can incorporate the pickup with the least additional travel time. For every assigned order, the algorithm inserts the pickup and dropoff points into the driver’s existing route at the positions that minimize extra distance, using APSP paths to maintain shortest travel between stops. Finally, total latency is computed by simulating each driver’s route, and the resulting assignment and route plan are returned as the solution.

Bibliography

- [1] **Yen's algorithm**. Link: https://en.wikipedia.org/wiki/Yen%27s_algorithm.
Accessed: 22 November 2025. Authors: Wikipedia contributors.